In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as mlt
```

```
Matplotlib is building the font cache; this may take a moment.
```

In [2]:
```
data = [
    ["ORD101", "2026-10-01", " Laptop ",     55000,    2,    " South ", "Electronic
    ["ORD102", "2026-10-02", "Mouse",          800,    5,    "NORTH",   "Accessorie
    ["ORD103", "2026-10-03", "Keyboard",      1500,    3,    " east ",  "Accessorie
    ["ORD104", "2026-10-04", "Monitor",      12000,    2,    "West",    "Electronic
    ["ORD105", "2026-10-05", "Laptop",        58000,    1,    "south",   "Electroni
    ["ORD106", "2026-10-06", "Headphones",     None,    4,    "North",   "Accessori
    ["ORD107", "2026-10-07", "Webcam",         2500, None,   "East",    "Accessorie
    ["ORD108", "2026-10-08", "Printer",        9000,    2,    None,      "Electroni
    ["ORD109", "2026-10-09", "USB Hub",        1200,    6,    "West",    None],
    ["ORD110", "2026-10-10", "Laptop",       650000,    1,    "South",   "Electroni
    ["ORD111", "2026-10-11", "Mouse",           800,    8,    "NORTH",   "Accessori
    ["ORD112", "2026-10-12", "Monitor",       12500,    2,    " east ",  "Electroni
    ["ORD113", "2026-10-13", "Keyboard",    "invalid",  4,    "West",    "Accessori
    ["ORD114", "2026-10-14", "Webcam",         2800,    3,    "South",   "Accessori
    ["ORD101", "2026-10-01", " Laptop ",     55000,    2,    " South ", "Electronic
]

columns = [
    "order_id", "order_date", "product",
    "price", "quantity", "region", "category"
]

df = pd.DataFrame(data, columns=columns)

df
```

Out[2]:

	order_id	order_date	product	price	quantity	region	category
0	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics
1	ORD102	2026-10-02	Mouse	800	5.0	NORTH	Accessories
2	ORD103	2026-10-03	Keyboard	1500	3.0	east	Accessories
3	ORD104	2026-10-04	Monitor	12000	2.0	West	Electronics
4	ORD105	2026-10-05	Laptop	58000	1.0	south	Electronics
5	ORD106	2026-10-06	Headphones	None	4.0	North	Accessories
6	ORD107	2026-10-07	Webcam	2500	NaN	East	Accessories
7	ORD108	2026-10-08	Printer	9000	2.0	NaN	Electronics
8	ORD109	2026-10-09	USB Hub	1200	6.0	West	NaN
9	ORD110	2026-10-10	Laptop	650000	1.0	South	Electronics
10	ORD111	2026-10-11	Mouse	800	8.0	NORTH	Accessories
11	ORD112	2026-10-12	Monitor	12500	2.0	east	Electronics
12	ORD113	2026-10-13	Keyboard	invalid	4.0	West	Accessories
13	ORD114	2026-10-14	Webcam	2800	3.0	South	Accessories
14	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics

In [4]:
```
df.head()
```

Out[4]:

	order_id	order_date	product	price	quantity	region	category
0	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics
1	ORD102	2026-10-02	Mouse	800	5.0	NORTH	Accessories
2	ORD103	2026-10-03	Keyboard	1500	3.0	east	Accessories
3	ORD104	2026-10-04	Monitor	12000	2.0	West	Electronics
4	ORD105	2026-10-05	Laptop	58000	1.0	south	Electronics

In [5]:
```
df.shape
```

Out[5]:
```
(15, 7)
```

In [6]:
```
df.columns
```

Out[6]:
```
Index(['order_id', 'order_date', 'product', 'price', 'quantity', 'region',
       'category'],
      dtype='str')
```

In [7]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   order_id    15 non-null     str
 1   order_date  15 non-null     str
 2   product     15 non-null     str
 3   price       14 non-null     object
 4   quantity    14 non-null     float64
 5   region      14 non-null     str
 6   category    14 non-null     str
dtypes: float64(1), object(1), str(5)
memory usage: 972.0+ bytes
```

In [8]:
```
df.describe()
```

Out[8]:

	quantity
count	14.000000
mean	3.214286
std	2.006856
min	1.000000
25%	2.000000
50%	2.500000
75%	4.000000
max	8.000000

In [9]:
```
df.isnull().sum()
```

Out[9]:
```
order_id      0
order_date    0
product       0
price         1
quantity      1
region        1
category      1
dtype: int64
```

Business impact of missing values

price - Revenue can be underreported or incorrectly calculated if the price is unavailable. quantity - The business cannot reliably determine units sold or order value. region - Regional sales reports may omit transactions or assign sales to the wrong location if missing data is handled incorrectly. category - Product-category reports may understate category sales or show incomplete product performance.

Example: If a customer orders four headphones but the price is missing, the company cannot calculate the exact order revenue from this dataset. It should verify the price against a reliable source or use an explicitly documented estimation rule.

In [41]:
```
df.duplicated().sum()
```

Out[41]:
```
np.int64(0)
```

In [11]:
```
df[df.duplicated(subset="order_id", keep=False)]
```

Out[11]:

	order_id	order_date	product	price	quantity	region	category
0	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics
14	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics

Business impact of duplicate orders

Suppose order ORD101 has a price of ₹55,000 and a quantity of 2.

Its revenue is: 55000 * 2 = 110000

If the same order is counted twice, the report could show ₹220,000 for that order. This would overstate revenue by ₹110,000.

Important: In real systems, do not delete records based on order ID alone unless the business rule guarantees that each order ID represents one complete order record. An order can legitimately contain multiple line items. Investigate the source and choose the correct deduplication key.

In [12]:
```
df = df.drop_duplicates(
    subset="order_id",
    keep="first"
).copy()

print("Shape after removing duplicates:", df.shape)
```

```
Shape after removing duplicates: (14, 7)
```

In [15]:
```
text_columns = ["order_id", "product", "region", "category"]

for col in text_columns:
    df[col] = df[col].astype("string").str.strip()

df["product"] = df["product"].str.title()
df["region"] = df["region"].str.title()
df["category"] = df["category"].str.title()

df[["product", "region", "category"]].head(10)
```

Out[15]:

	product	region	category
0	Laptop	South	Electronics
1	Mouse	North	Accessories
2	Keyboard	East	Accessories
3	Monitor	West	Electronics
4	Laptop	South	Electronics
5	Headphones	North	Accessories
6	Webcam	East	Accessories
7	Printer	<NA>	Electronics
8	Usb Hub	West	<NA>
9	Laptop	South	Electronics

In [16]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   order_id    14 non-null     string
 1   order_date  14 non-null     str
 2   product     14 non-null     string
 3   price       13 non-null     object
 4   quantity    13 non-null     float64
 5   region      13 non-null     string
 6   category    13 non-null     string
dtypes: float64(1), object(1), str(1), string(4)
memory usage: 916.0+ bytes
```

In [17]:
```
df["order_date"] = pd.to_datetime(
    df["order_date"],
    errors="coerce"
)

df["order_date"].dtype
```

Out[17]:
```
dtype('<M8[us]')
```

In [18]:
```
df.head()
```

Out[18]:

	order_id	order_date	product	price	quantity	region	category
0	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics
1	ORD102	2026-10-02	Mouse	800	5.0	North	Accessories
2	ORD103	2026-10-03	Keyboard	1500	3.0	East	Accessories
3	ORD104	2026-10-04	Monitor	12000	2.0	West	Electronics
4	ORD105	2026-10-05	Laptop	58000	1.0	South	Electronics

In [19]:
```
df
```

Out[19]:

	order_id	order_date	product	price	quantity	region	category
0	ORD101	2026-10-01	Laptop	55000	2.0	South	Electronics
1	ORD102	2026-10-02	Mouse	800	5.0	North	Accessories
2	ORD103	2026-10-03	Keyboard	1500	3.0	East	Accessories
3	ORD104	2026-10-04	Monitor	12000	2.0	West	Electronics
4	ORD105	2026-10-05	Laptop	58000	1.0	South	Electronics
5	ORD106	2026-10-06	Headphones	None	4.0	North	Accessories
6	ORD107	2026-10-07	Webcam	2500	NaN	East	Accessories
7	ORD108	2026-10-08	Printer	9000	2.0	<NA>	Electronics
8	ORD109	2026-10-09	Usb Hub	1200	6.0	West	<NA>
9	ORD110	2026-10-10	Laptop	650000	1.0	South	Electronics
10	ORD111	2026-10-11	Mouse	800	8.0	North	Accessories
11	ORD112	2026-10-12	Monitor	12500	2.0	East	Electronics
12	ORD113	2026-10-13	Keyboard	invalid	4.0	West	Accessories
13	ORD114	2026-10-14	Webcam	2800	3.0	South	Accessories

In [20]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   order_id    14 non-null     string
 1   order_date  14 non-null     datetime64[us]
 2   product     14 non-null     string
 3   price       13 non-null     object
 4   quantity    13 non-null     float64
 5   region      13 non-null     string
 6   category    13 non-null     string
dtypes: datetime64[us](1), float64(1), object(1), string(4)
memory usage: 916.0+ bytes
```

In [21]:
```
df["price"] = pd.to_numeric(
    df["price"],
    errors="coerce"
)

df["price"].dtype
```

Out[21]:
```
dtype('float64')
```

In [22]:
```
df["quantity"] = pd.to_numeric(
    df["quantity"],
    errors="coerce"
)

df["quantity"].dtype
```

Out[22]:
```
dtype('float64')
```

Business impact of incorrect data types

- Price stored as text: Numerical calculations and price comparisons may fail.
- Date stored as text: Monthly trends, date filtering, and time-based analysis become less reliable.
- Quantity stored as text: Total units sold cannot be calculated reliably. Converting these columns to appropriate data types allows the business to calculate revenue, compare prices, and analyze sales by date.

In [23]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 14 entries, 0 to 13
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   order_id    14 non-null     string
 1   order_date  14 non-null     datetime64[us]
 2   product     14 non-null     string
 3   price       12 non-null     float64
 4   quantity    13 non-null     float64
 5   region      13 non-null     string
 6   category    13 non-null     string
dtypes: datetime64[us](1), float64(2), string(4)
memory usage: 916.0 bytes
```

In [24]:
```
df.isnull().sum()
```

Out[24]:
```
order_id      0
order_date    0
product       0
price         2
quantity      1
region        1
category      1
dtype: int64
```

The missing-value counts should now be zero for these columns.

Business impact of missing-value handling

- Filling a missing price with the median allows calculations to proceed, but the resulting revenue is an estimate, not necessarily the actual revenue.
- Filling missing quantity with the median allows analysis to proceed, but it can distort unit-sales totals.
- Replacing a missing region with "Unknown" preserves the transaction without falsely assigning it to a known region.
- Replacing a missing category with "Unknown" preserves the transaction for overall sales reporting while keeping its category unresolved. Real-world recommendation: For financial reports, use verified transaction values whenever possible. If imputed values are used for analytical purposes, flag them so that estimated revenue is not confused with actual billed revenue.

In [25]:
```
price_median = df["price"].median()
quantity_median = df["quantity"].median()

df["price"] = df["price"].fillna(price_median)

df["quantity"] = df["quantity"].fillna(
    round(quantity_median)
)
```

```
df["region"] = df["region"].fillna("Unknown")
df["category"] = df["category"].fillna("Unknown")

df.isnull().sum()
```

Out[25]:
```
order_id      0
order_date    0
product       0
price         0
quantity      0
region        0
category      0
dtype: int64
```

In [26]:
```
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)

print("Q1:", Q1)
print("Q3:", Q3)
```

```
Q1: 1750.0
Q3: 12375.0
```

In [27]:
```
IQR = Q3 - Q1

print("IQR:", IQR)
```

```
IQR: 10625.0
```

In [28]:
```
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Lower bound:", lower_bound)
print("Upper bound:", upper_bound)
```

```
Lower bound: -14187.5
Upper bound: 28312.5
```

In [29]:
```
outliers = df[
    (df["price"] < lower_bound) |
    (df["price"] > upper_bound)
]

outliers[["order_id", "product", "price"]]
```

Out[29]:

	order_id	product	price
0	ORD101	Laptop	55000.0
4	ORD105	Laptop	58000.0
9	ORD110	Laptop	650000.0

Business impact of outliers

A price of ₹650,000 for a laptop may be legitimate, or it may be a data-entry error.

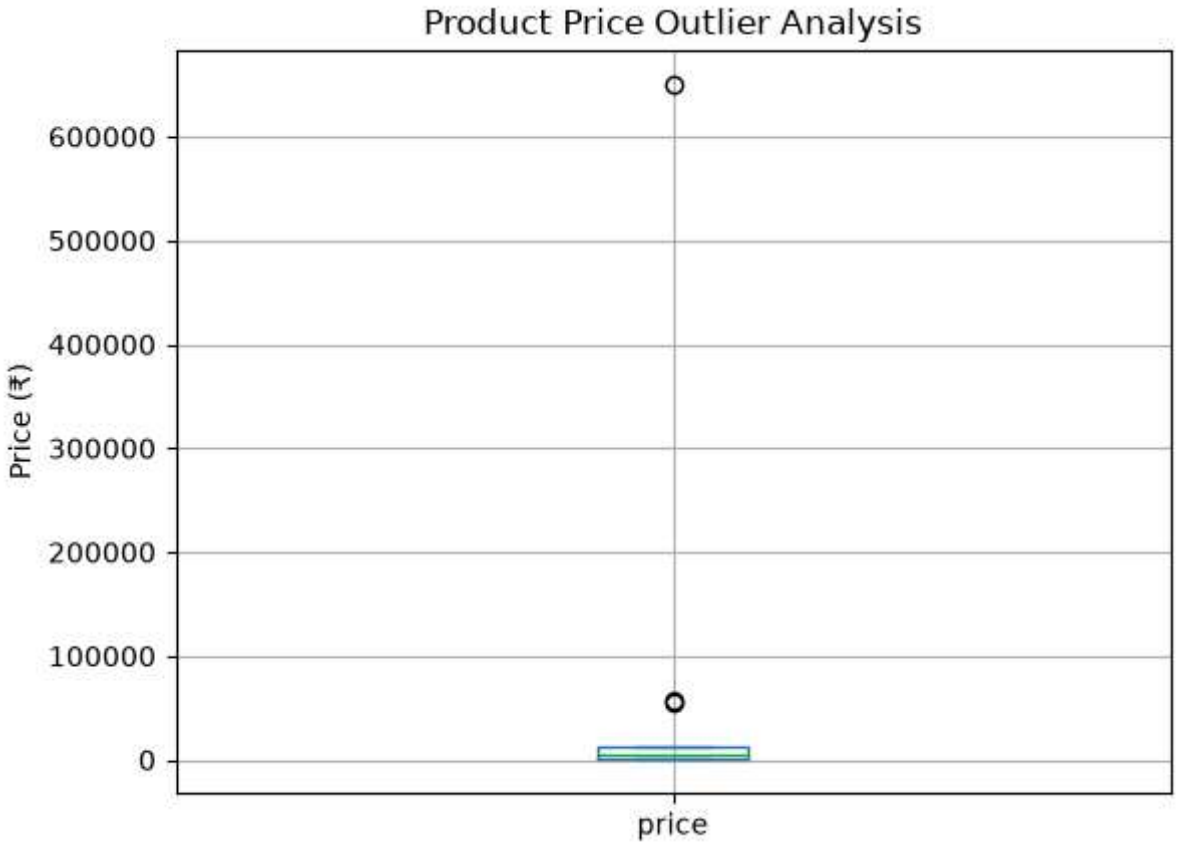
If the price is incorrect:

- Average product prices may become misleading.
- Revenue forecasts may be distorted.
- Management may incorrectly conclude that high-value products are performing exceptionally well.
- Inventory and pricing decisions may be based on incorrect figures. However, an outlier is not automatically an error. Flag it, investigate it, and only correct or remove it when there is evidence that the value is wrong.

In [30]:
```
df["price"].plot(kind="box")

mlt.title("Product Price Outlier Analysis")
mlt.ylabel("Price (₹)")
mlt.grid(True)

mlt.show()
```



In [31]:
```
df["revenue"] = df["price"] * df["quantity"]

df[["order_id", "product", "price", "quantity", "revenue"]]
```

Out[31]:

	order_id	product	price	quantity	revenue
0	ORD101	Laptop	55000.0	2.0	110000.0
1	ORD102	Mouse	800.0	5.0	4000.0
2	ORD103	Keyboard	1500.0	3.0	4500.0
3	ORD104	Monitor	12000.0	2.0	24000.0
4	ORD105	Laptop	58000.0	1.0	58000.0
5	ORD106	Headphones	5900.0	4.0	23600.0
6	ORD107	Webcam	2500.0	3.0	7500.0
7	ORD108	Printer	9000.0	2.0	18000.0
8	ORD109	Usb Hub	1200.0	6.0	7200.0
9	ORD110	Laptop	650000.0	1.0	650000.0
10	ORD111	Mouse	800.0	8.0	6400.0
11	ORD112	Monitor	12500.0	2.0	25000.0
12	ORD113	Keyboard	5900.0	4.0	23600.0
13	ORD114	Webcam	2800.0	3.0	8400.0

In [32]:
```
def classify_revenue(revenue):
    if revenue >= 50000:
        return "High"
    elif revenue >= 10000:
        return "Medium"
    else:
        return "Low"
```

In [33]:
```
df["revenue_category"] = df["revenue"].apply(
    classify_revenue
)

df[["order_id", "revenue", "revenue_category"]]
```

Out[33]:

	order_id	revenue	revenue_category
0	ORD101	110000.0	High
1	ORD102	4000.0	Low
2	ORD103	4500.0	Low
3	ORD104	24000.0	Medium
4	ORD105	58000.0	High
5	ORD106	23600.0	Medium
6	ORD107	7500.0	Low
7	ORD108	18000.0	Medium
8	ORD109	7200.0	Low
9	ORD110	650000.0	High
10	ORD111	6400.0	Low
11	ORD112	25000.0	Medium
12	ORD113	23600.0	Medium
13	ORD114	8400.0	Low

In [34]:
```
df
```

Out[34]:

	order_id	order_date	product	price	quantity	region	category	revenue
0	ORD101	2026-10-01	Laptop	55000.0	2.0	South	Electronics	110000.0
1	ORD102	2026-10-02	Mouse	800.0	5.0	North	Accessories	4000.0
2	ORD103	2026-10-03	Keyboard	1500.0	3.0	East	Accessories	4500.0
3	ORD104	2026-10-04	Monitor	12000.0	2.0	West	Electronics	24000.0
4	ORD105	2026-10-05	Laptop	58000.0	1.0	South	Electronics	58000.0
5	ORD106	2026-10-06	Headphones	5900.0	4.0	North	Accessories	23600.0
6	ORD107	2026-10-07	Webcam	2500.0	3.0	East	Accessories	7500.0
7	ORD108	2026-10-08	Printer	9000.0	2.0	Unknown	Electronics	18000.0
8	ORD109	2026-10-09	Usb Hub	1200.0	6.0	West	Unknown	7200.0
9	ORD110	2026-10-10	Laptop	650000.0	1.0	South	Electronics	650000.0
10	ORD111	2026-10-11	Mouse	800.0	8.0	North	Accessories	6400.0
11	ORD112	2026-10-12	Monitor	12500.0	2.0	East	Electronics	25000.0
12	ORD113	2026-10-13	Keyboard	5900.0	4.0	West	Accessories	23600.0
13	ORD114	2026-10-14	Webcam	2800.0	3.0	South	Accessories	8400.0

In [35]:

```
region_revenue = (
    df.groupby("region")["revenue"]
      .sum()
      .sort_values(ascending=False)
)

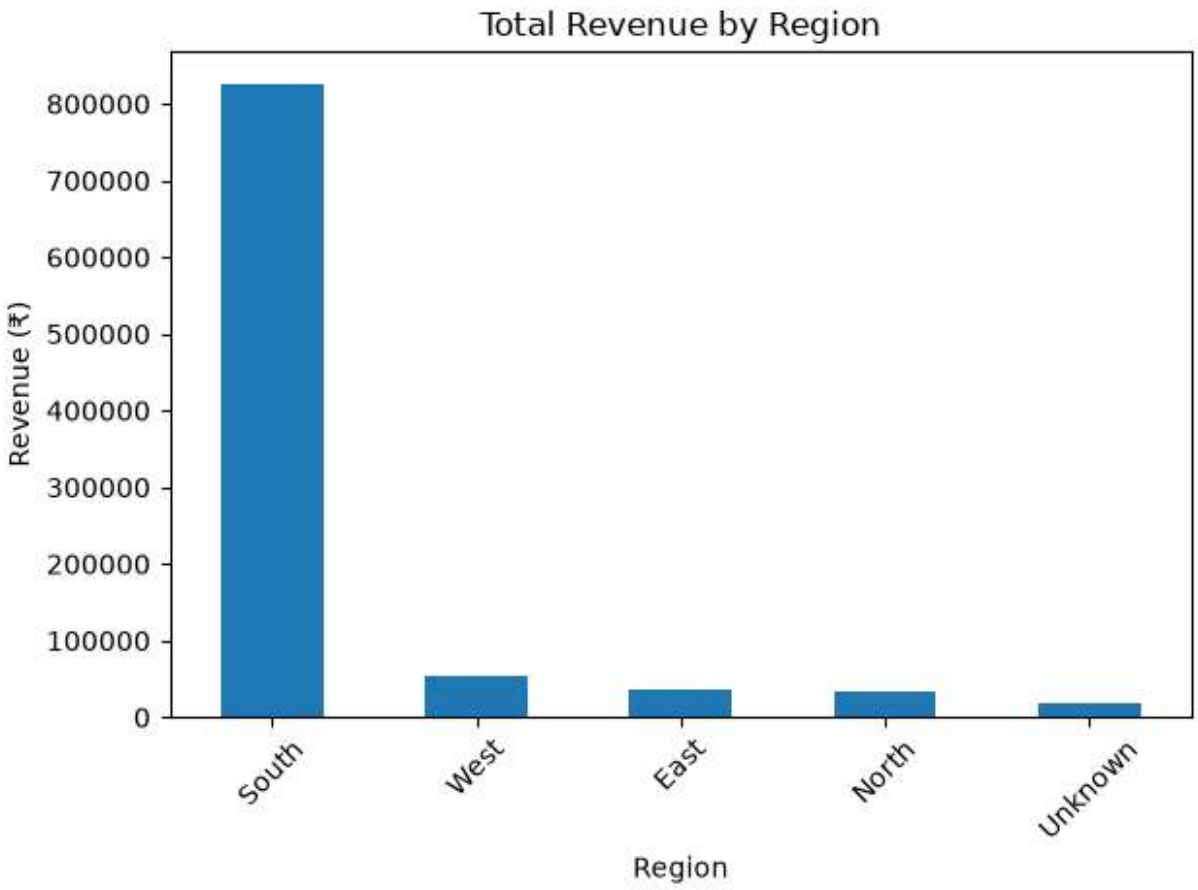
region_revenue
```

Out[35]:
```
region
South      826400.0
West        54800.0
East        37000.0
North       34000.0
Unknown     18000.0
Name: revenue, dtype: float64
```

In [36]:
```
region_revenue.plot(kind="bar")

mlt.title("Total Revenue by Region")
mlt.xlabel("Region")
mlt.ylabel("Revenue (₹)")
mlt.xticks(rotation=45)
mlt.tight_layout()

mlt.show()
```



In [42]:
```
print("Dataset shape:", df.shape)

print("\nMissing values:")
print(df.isnull().sum())

print("\nDuplicate order IDs:")
print(df["order_id"].duplicated().sum())

print("\nData types:")
print(df.dtypes)
```

```
display(df.head())
```

```
Dataset shape: (14, 9)

Missing values:
order_id            0
order_date          0
product             0
price               0
quantity            0
region              0
category            0
revenue             0
revenue_category    0
dtype: int64

Duplicate order IDs:
0

Data types:
order_id                    string
order_date          datetime64[us]
product                     string
price                      float64
quantity                   float64
region                      string
category                    string
revenue                    float64
revenue_category               str
dtype: object
```

	order_id	order_date	product	price	quantity	region	category	revenue	revenue_
0	ORD101	2026-10-01	Laptop	55000.0	2.0	South	Electronics	110000.0	
1	ORD102	2026-10-02	Mouse	800.0	5.0	North	Accessories	4000.0	
2	ORD103	2026-10-03	Keyboard	1500.0	3.0	East	Accessories	4500.0	
3	ORD104	2026-10-04	Monitor	12000.0	2.0	West	Electronics	24000.0	
4	ORD105	2026-10-05	Laptop	58000.0	1.0	South	Electronics	58000.0	

In [43]:
```
df.to_csv("cleaned_sales_data.csv", index=False)

print("Cleaned sales dataset exported successfully!")
```

```
Cleaned sales dataset exported successfully!
```

In [ ]: